

SDK / стек

Параметр	Значение
Название приложения	Betano'
Android Gradle Plugin	8.7.2
minSdk	24
targetSdk	35
Kotlin	да + 1.9.0
Web View	нет
Custom Tabs	нет
Рекламные сети	нет
Аналитика	Firebase Analytics
Permissions	com.google.android.gms.permission.AD_ID, android.permission.INTERNET, android.permission.ACCESS_NETWORK_STATE, android.permission.WAKE_LOCK, com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE, android.permission.ACCESS_AD SERVICES_ATTRIBUTION, android.permission.ACCESS_AD SERVICES_AD_ID, com.gamified.taskplanner.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.android.vending.CHECK_LICENSE
Libraries	Unity 6000.0.60f1 (IL2CPP), Firebase Analytics 23.2.0, Firebase Installations 17.0.1, Firebase Unity SDK 13.10.0 (Analytics/App/Remote Config/Storage export), play-services-ads-identifier 18.0.0, play-services-base 18.10.0, play-services-basement 18.10.0, play-services-tasks 18.4.1, play-services-measurement 23.2.0, AndroidX (core/core-ktx 1.9.0, datastore 1.1.7, activity, fragment, lifecycle, privacysandbox ads), kotlinx-coroutines 1.9.0, MiniIT AdvertisingIdFetcher, com.pairip.licensecheck, UniTask, DOTween, Unity Addressables, bitter.jnibridge, okio, Guava
Подозрительные домены	gamified-task-planner.firebaseiostorage.app
SharedPreferences	Unity PlayerPrefs: ключ referrer / bc12bafaadc238 (данные установки), ключи isSoundMuted / isMusicMuted; SaveLoadService хранит GameData белого приложения
Есть ли клоака	да
Подозрительные слова	UpdateTool, RequestParams, VerifScreen, AdvertisingId, referrer, OpenURL, RemoteConfig, bridge, license check, ParserLinkAuthority, FlagLinkVisitor, ClickID

Проверка домена: gamified-task-planner.firebaseiostorage.app

Параметр / движок	Значение / вердикт
Домен	gamified-task-planner.firebaseiostorage.app
VirusTotal	ошибка: домен не найден в VirusTotal
Куда редиректит	нет (DNS не резолвится)
Что выводит (кратко)	нет (DNS не резолвится)
Где припаркован	нет

Какие данные собираются

- идентификатор AppsFlyer
- рекламный номер устройства
- модель телефона
- версия системы
- идентификатор игрока
- номер выпуска Android
- идентификатор сборки
- версия приложения
- реферер установки
- время установки

Как собираются

Сразу после запуска Unity-игры, ещё до обычного экрана планировщика задач, встроенный модуль UpdateTool сам читает сведения об устройстве. Человек этого обычно не замечает: отдельного окна с разрешением на этот сбор нет.

Рекламный номер устройства берут через Android Advertising Id (класс AndroidAdvertisingIdHelper и пакет MiniIT AdvertisingIdFetcher). Модель телефона и версию системы читают из сведений Android. Реферер установки и время установки запрашивают у службы Google Play (Install Referrer): в коде есть GetReferrer, GetInstallTime, GetInstallInfo и обработчики onInstallReferrerSetupFinished.

Все эти значения складывают в объект RequestParams с типами RequestParameter (AppsFlyerId, AdvertisingId, DeviceModel, OsVersion, PlayerId, OSVersionRelease, BuildID, AppVersion, Referrer, InstallTime). Для сетевого тела рядом явно видны поля os_version и device_model.

Куда отправляются

Собранные сведения уходят в инфраструктуру Firebase проекта gamified-task-planner. В приложении прописано хранилище gamified-task-planner.firebaseio.com (строка google_storage_bucket и google-services.json). Модуль TagCollectKey97 лежит в UpdateTool.CodeBase.Components.FireBase — то есть проверка завязана на Firebase.

Перед запросом приложение поднимает служебные токены Firebase (App Check и Auth) через HttpHelpers.SetRequestHeaders и потом проверяет ответ ValidateHttpResponse. Точный полный URL собран из обфусцированных кусков рядом с RequestParams (строки вроде wvqqch7bplpujgr, harop5o и другие), поэтому открытым текстом в коде он не лежит, но направление — Firebase Storage / HTTP к этому проекту.

Запрос уходит тихо в фоне при старте, пока человеку показывают загрузку или ещё не открыли обычный планировщик задач. Запасных «чужих» доменов вне этого Firebase-проекта в открытом виде не видно.

Как фильтруются пользователи

На проверку уходят рекламный номер устройства, модель телефона, версии системы и приложения, реферер установки и время установки. По этим признакам сервер может отличать живой трафик магазина от «грязного», смотреть источник установки и решать, кому дать внешнюю ссылку.

В самом приложении на телефоне нет явного списка стран или языков «кого пускать». Локально видны только сбор признаков и разбор ответа. Решение «кому оффер, кому обычное приложение» делает удалённая сторона (Firebase / ответ HTTP).

Дополнительный сигнал схемы: в Google Play приложение называется Betano', а внутри APK label и

productName — Your Task Planner. Это белая оболочка под другое имя.

Что возвращается

Ответ обрабатывают классы ParserLinkAuthority и FlagLinkVisitor10: по их именам и месту в цепочке видно, что из ответа разбирают внешнюю ссылку и затем её открывают. Рядом идут ViewAuthorizeComponent и VerifScreen — шаг проверки, после которого решают, пускать ли человека дальше.

Имена отдельных полей ответа в открытом виде спрятаны обфускацией UpdateTool. По логике модулей видно два исхода: есть ссылка для открытия — показывают внешнюю страницу; ссылки нет или проверка не пройдена — остаётся обычный режим Your Task Planner.

Также в цепочке есть ValidateHttpResponse и GetStatusCode: приложение смотрит, успешно ли прошёл служебный ответ по сети, прежде чем открывать ссылку.

Как показывается оффер или белая версия

Если проверка вернула боевую ссылку, её открывают во внешнем браузере телефона (системный вызов Unity Application.OpenURL), а не во встроенном окне сайта внутри приложения. Отдельного встроенного окна сайта и открытия через Chrome Custom Tabs в коде показа оффера не видно. Что именно на странице (букмекер вроде Betano или другой рекламный сайт), решает удалённый ответ; в APK адрес оффера заранее не прописан открытым текстом.

Если ссылки нет или сервер не пускает на оффер, человеку просто остаётся обычное приложение Your Task Planner без перехода во внешний браузер.